

MIDDLEEARTH RAG

An MLOps Pipeline for a Lord of the Rings RAG System

Marcos Nepomuceno Costa

Machine Learning and Data in Operations (MLOps), ZHAW MSE, Spring 2026

11.04.2026



Project Overview

- **What:** End-to-end RAG system that answers questions about *The Lord of the Rings* , wrapped in a production MLOps pipeline
- **Why RAG:** Focus is on the ops — data versioning, retrieval evaluation, model registry, deployment, monitoring, and human feedback — without fine-tuning cost
- **Why LOTR:** Bounded, well-known corpus; easy to sanity-check answers; rich enough to produce interesting retrieval failures worth measuring
- **Data:** ~7,000 articles from LOTR Fandom Wiki + Wikipedia Tolkien category via MediaWiki API
-

Pipeline:



Components and Tools

DVC + Git

Data Versioning & Artifact Storage

mlflow

Experiment Tracking & Model Registry

PyTorch + HuggingFace

Embeddings & LLM Inference

Docker + FastAPI + Streamlit

Deployment & User Interface

GitHub Actions

CI/CD

Label Studio

Human-in-the-Loop Feedback

Data Versioning & Artifact Storage

DVC ♦ git

- **Reproducibility:** DVC versions the LOTR corpus, chunked Parquet files, FAISS index, and gold eval set alongside Git-managed code — every experiment is fully reproducible
- **Storage Efficiency:** Large data files stored externally; Git repository stays lightweight with only `.dvc` pointer files committed
- **Pipeline Automation:** `dvc repro` re-runs only stages whose inputs changed — chunk size, overlap, and embedding model all parameterized in `params.yaml`

Experiment Tracking & Model Registry

mlflow

- **Centralized Tracking:** Every eval run logs retrieval metrics (Recall@k, MRR) and generation metrics (faithfulness, answer relevance) to a local MLflow server
- **Model Registry:** Embedding model versions, prompt templates, and index versions registered as MLflow artifacts — governance over development, staging, and production
- **CI Gate:** Mini eval (10-sample subset) runs on every PR; build fails if Recall@5 drops below the defined threshold

Deployment & CI/CD

[Docker](#) · [FastAPI](#) · [Streamlit](#) · [GitHub Actions](#)

- **One-command deployment:** `docker compose up` brings up API, Streamlit UI, MLflow tracking server, and Label Studio — full stack reproducible on any machine
- **FastAPI:** `/predict` , `/liveness` , `/readiness` endpoints; serves ONNX-exported embedding model for fast CPU inference in the container
- **Streamlit UI:** Chat interface showing answers + retrieved source chunks + thumbs up/down feedback button
- **GitHub Actions:** Automated pytest + mini RAG eval on every push; Docker image build validation; blocks merges on retrieval regressions

Human-in-the-Loop Feedback

Label Studio

- **Feedback Collection:** Streamlit UI collects thumbs up/down ratings on every answer; low-confidence responses automatically queued for human review in Label Studio
- **Review Interface:** Human annotators mark the correct retrieved chunk and can rewrite poor answers — structured via a custom Label Studio XML config
- **Closing the Loop:** Reviewed ratings feed back into the gold eval set, versioned with DVC, ready for the next iteration — the MLOps flywheel in action